By Turnitin™

Form 3 - ERP Based Inventory Monitoring Framework for Three Month Demand Forecasting Using Supervised Machine Learnin...

Kelas Kimia 014
Kelas Kimia
Universitas Pendidikan Muhammadiyah Sorong

Document Details

Submission ID
trn:oid:::1:3524552941

Submission Date
Apr 1, 2026, 10:42 PM GMT+7

Download Date
Apr 1, 2026, 10:49 PM GMT+7

File Name
Form_3_-_ERP_Based_Inventory_Monitoring_Framework_for_Three_Month_Demand_Forecasting....docx

File Size
4.4 MB

18 Pages

2,862 Words

17,259 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

9 Not Cited or Quoted 6%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
1% Publications
4% Submitted works (Student Papers)

Match Groups

- **9** Not Cited or Quoted 6%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
1% Publications
4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
repository.president.ac.id		4%
2	Publication	
Arvind Dagur, Sohit Agarwal, Dhirendra Kumar Shukla, Shabir Ali, Sandhya Sharm...		<1%
3	Student papers	
i-CATS University College		<1%
4	Internet	
www.mdpi.com		<1%
5	Internet	
zenodo.org		<1%
6	Internet	
ijasce.org		<1%

Form 3
Design Title: ERP Based Inventory Monitoring Framework for Three Month Demand Forecasting Using Supervised Machine Learning

GROUP MEMBER:

No.	Student Name	Student ID
1.	Adam Abdurrahman	012202300143
2.	Belva Tabitha	012202300108

Advisor: Rikip Ginanjar, M. Sci

Submitted for
Capstone Design Project
to Faculty of Computer Science
President University

TABLE OF CONTENT

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

Name : 1. Adam Abdurrahman – 012202300143

2. Belva Tabitha – 012202300108

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled "**ERP Based Inventory Monitoring Framework for Three Month Demand Forecasting Using Supervised Machine Learning**" is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, January 2026

Signer 1	Signer 2
	
Adam Abdurrahman – 012202300143	Belva Tabitha – 012202300108

SCREENSHOT OF ZEROGPT

PART 3: DESIGN

A. SYSTEM DESIGN

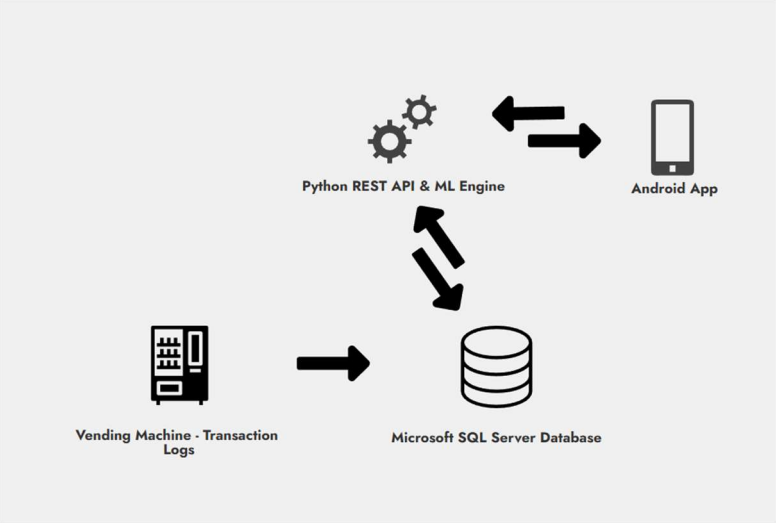

Figure 1.1 – Diagram Architecture

1. System Architecture Overview

The developed system utilizes a Client-Server architecture designed to integrate Internet of Things (IoT) transaction logs, a mobile-based Enterprise Resource Planning (ERP) interface, and a Supervised Machine Learning engine. The architecture ensures that General Affairs (GA) staff can monitor milk inventory in real-time and make data-driven procurement decisions based on 3-month demand forecasting.

The system architecture consists of five primary components:

- **Data Source (IoT Node):** The milk vending machines act as the primary data generators. Every dispense event (including timestamp, RFID, milk flavor, and slot number) is recorded as raw transaction logs.
- **Client Tier (Mobile Application):** A native Android application developed using Java. This serves as the primary user interface for GA Staff (to view operational dashboards, analyze forecasts, and create procurement plans) and IT Admins (to manage master data and configure user access).
- **Application/API Tier (Backend):** A Python-based REST API acts as the secure intermediary between the mobile application and the database. It handles data aggregation, business logic validation, and request routing.
- **Analytical/ML Tier:** A dedicated Python Machine Learning engine. It processes historical transaction data pulled from the database, runs the supervised learning algorithms, and returns total and flavor-level demand forecasts for the next three months.
- **Data Storage Tier:** Microsoft SQL Server is used as the centralized relational database. It securely stores master data (employees, vending machines, milk variants), aggregated transaction logs, forecast outputs, and finalized procurement decision records.

2. Connection and Data Flow

- Raw logs are extracted from the vending machine and imported into the SQL Server database.
- A System Scheduler (e.g., Cron Job) automatically triggers the Machine Learning Engine on a predefined schedule (e.g., the 1st of every month). The ML engine queries historical consumption data from the database, processes the predictions, calculates performance metrics (e.g., MAE, MAPE), and securely stores the generated forecast results back into the SQL Server database.
- The Android Application sends an HTTP request to the Python REST API when a user accesses the dashboard or views the 3-month forecast menu.
- The API retrieves these pre-calculated prediction results from the database, formats them into standard JSON format, and transmits them back to the Android application to be rendered as interactive visual charts.
- Once the GA Staff reviews and approves a draft procurement plan, the decision record is routed back through the API and permanently committed to the SQL Server as an official ERP operational record.

3. User Interface Mock-ups

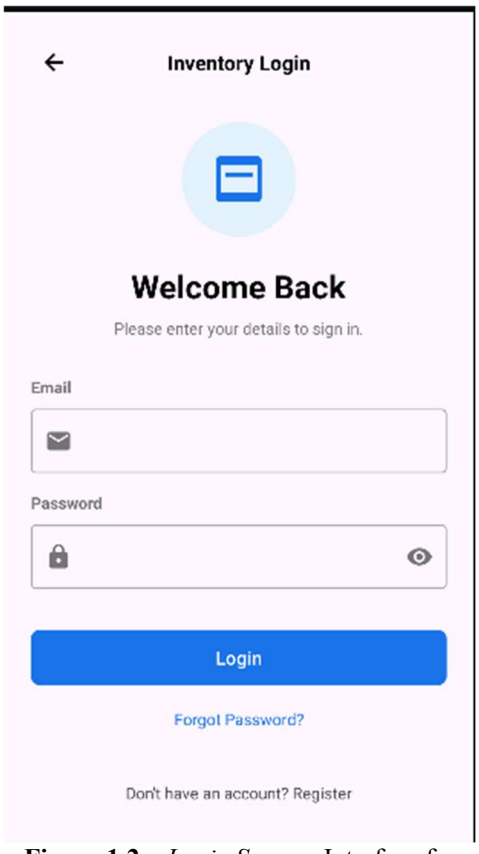

Figure 1.2 – *Login Screen:* Interface for secure user authentication with role-based access control.

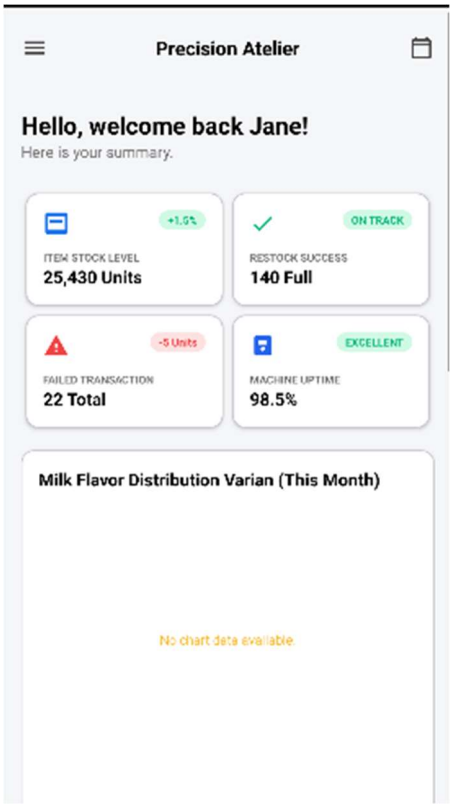

Figure 1.3 – *Main Dashboard:* Real-time visualization of current stock levels, daily consumption trends, and shift-based usage.

Figure 1.4 – *Forecast Module:* Displays the supervised machine learning prediction charts, flavor-level breakdown, and forecast accuracy metrics.

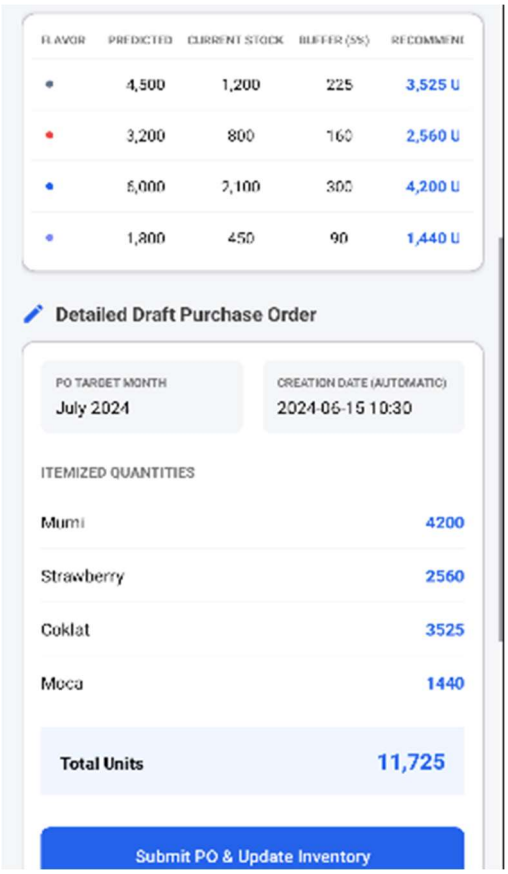

Figure 1.5 – *Procurement Planning:* The auto-generated ERP decision-support form recommending order quantities based on the forecasted demand.

B. HIERARCHICAL/ITERATIVE DESIGN

This section details the system's architecture from the highest level of user interaction down to the primitive software functions, utilizing standard UML modeling and defining the interfaces between each component.

1. System Modeling (Highest to Lowest Level)

a. *Highest Level:* At the highest design level, the system architecture is mapped through a Use Case Diagram (Figure 2.1) which illustrates the system boundaries and interactions of the main actors, namely GA Staff and IT Admin. As has been analyzed in detail in the previous document, this diagram summarizes the main operational functions of the application, ranging from dashboard monitoring, executing Machine Learning predictions for the next 3 months, to the process of creating and storing the Procurement Plan into the ERP system.

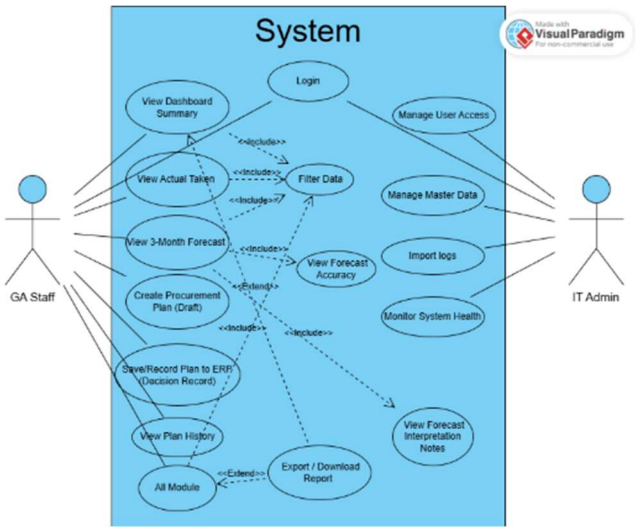

Figure 2.1 - *Use Case Diagram:* Defines the boundaries of the system and the authorized actions for each role, such as viewing forecasts, managing master data, and validating procurement plans.

Although the Use Case Diagram provides an overall view of 'what' the system and users can do, this diagram does not explain the logic sequence behind the scenes. Therefore, to understand 'how' these main operational functions are executed chronologically step by step, this high-level interaction will be broken down and derived to the mid-level through the elaboration of the Activity Diagram in the following section.

b. *Mid-Level:* Moving down one level (Mid-Level), the system's architecture is first mapped through a Context Diagram (Figure 2.2) to establish the overarching data flow between the external entities (GA Staff, IT Admin, and the Vending Machine) and the core ERP system. This diagram serves as a crucial bridge, illustrating the high-level input-output relationships before the system is decomposed into specific operational steps.

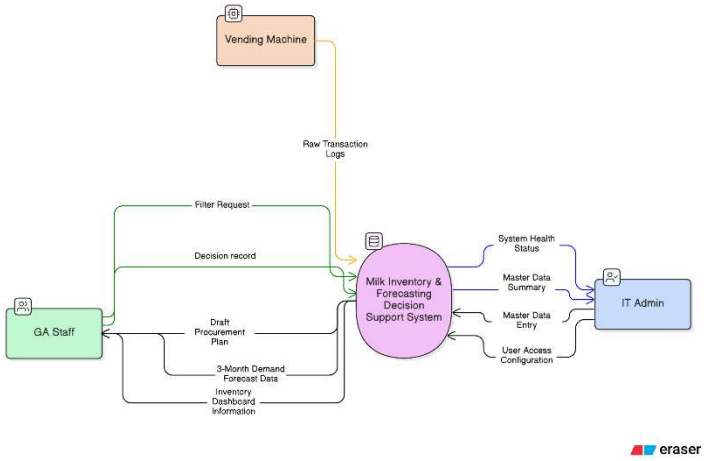

Figure 2.2 - Context Diagram of the Systems

From this generalized data flow, the system's operational logic is further detailed into specific business workflows using Activity Diagrams. The first crucial process elaborated is the Machine Learning forecasting execution (Figure 2.3). This diagram highlights the step-by-step validation of historical data adequacy, detailing how the system handles sufficient versus insufficient transaction logs before the algorithm computation is performed.

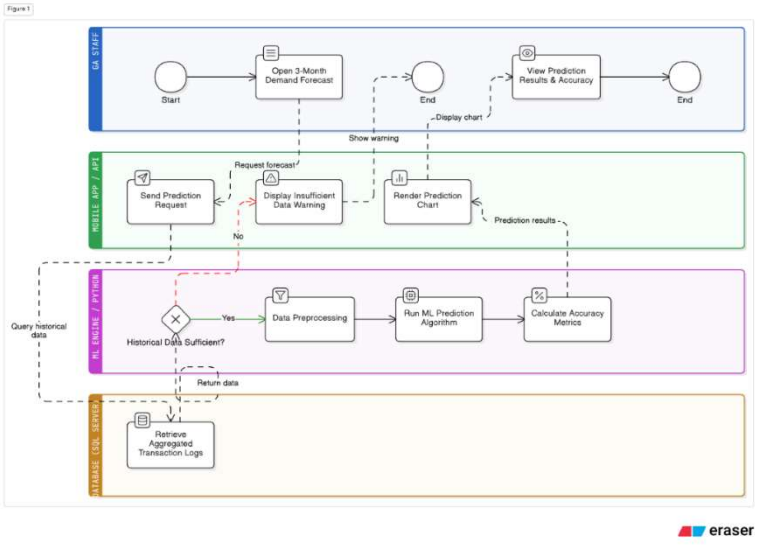

Figure 2.3 – Machine Learning Forecasting Activity

Once the automated forecasting process generates a prediction, the workflow transitions into the ERP decision-making phase. Figure 2.4 details the Procurement Validation Activity, where the ML prediction results are translated into a draft procurement plan. This workflow illustrates how the GA Staff can validate, adjust, and permanently store the finalized plan, effectively transforming automated machine learning predictions into structured managerial actions.

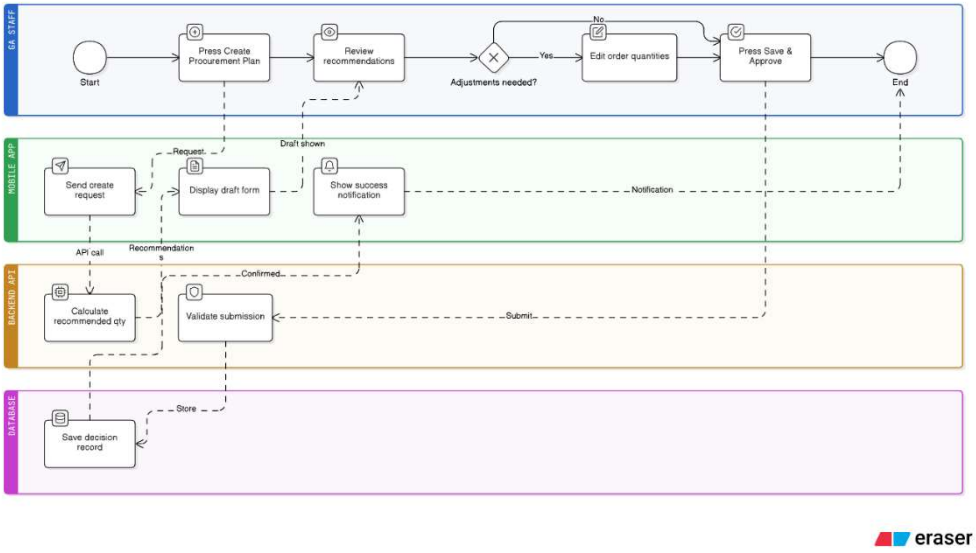

Figure 2.4 - Procurement Validation Activity

While these Activity Diagrams have successfully mapped the business workflow logic comprehensively, they do not yet show how the software modules communicate with each other technically from second to second. To observe the precise details of data exchange, API calls, and background script executions chronologically, the system design is further broken down to its lowest level through a Sequence Diagram in the following section.

c. *Lowest Level:* Primitive Software Functions (Sequence Diagram). At the lowest design level, system operations are broken down into primitive software functions that focus on technical interactions between components. Through the Sequence Diagram (Figure 2.4), it is mapped precisely how the System Scheduler automatically triggers the Machine Learning Engine in the background to process data from the SQL Database, as well as how the Android App and Python REST API exchange messages via the HTTP protocol when GA Staff access the interface. This diagram validates the separation of heavy computational load from application response time, ensuring the system's performance remains optimal.

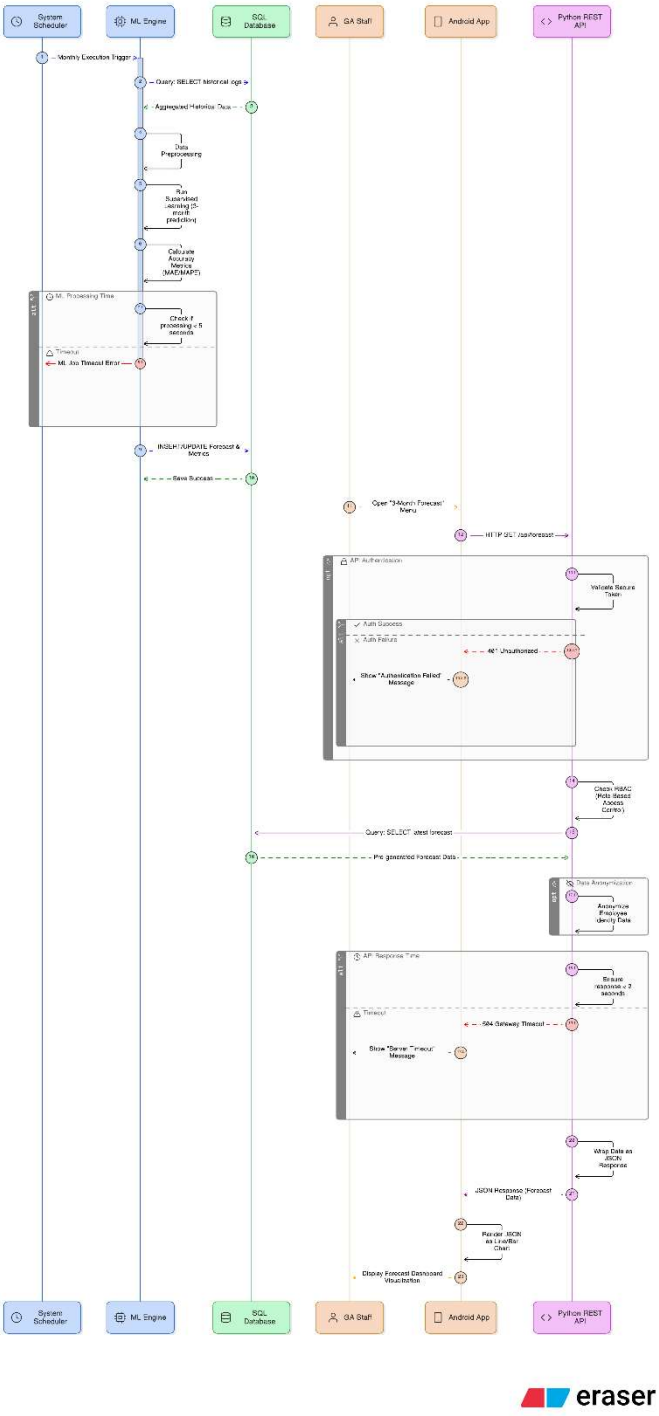

Figure 2.5 - System Sequence Diagram

Considering the complexity of message exchanges and parameters between application tiers depicted in the Sequence Diagram, a robust communication protocol and data storage foundation are required so that the system can be integrated seamlessly. Therefore, the following section will specifically define the interface mechanism (Interface Information) that bridges these components, as well as present the database architecture (Data Structure Modeling) which serves as the main pillar of system storage.

2. Interface Information Between Block Diagrams

To ensure seamless communication across the Client, API, and Database tiers, the system utilizes specific interface mechanisms:

- **Message Passing (Client to Server):** The Android mobile application communicates with the Python Backend over the network using HTTP REST API protocols. Requests are sent via HTTP GET/POST methods, and the server responds by passing messages formatted in standard JSON strings. For example, when requesting the 3-month forecast, the Android app sends an HTTP GET request to the /api/forecast endpoint, and the Python server returns a JSON payload such as:
  {"status": "success", "flavor_predictions": {"Murni": 4200, "Coklat": 3525, "Strawberry": 2560}}.
- **Parameter Passing (Internal Backend):** Within the Python Backend, data is exchanged between modules using strict parameter passing to maintain function isolation. For instance, when the System Scheduler triggers the ML forecasting script, it passes specific execution parameters to define the prediction scope (e.g., calling the function def run_forecast(target_month, execution_date, historical_limit)) to the Python processing functions.
- **Database Interfacing:** The Python backend (both the REST API and the ML Engine) connects to the Microsoft SQL Server using standard connection strings and executes parameterized SQL queries to ensure security against SQL injection. Specifically, the ML Engine securely reads historical logs by querying the monitor_log_datatransaksi table and writes forecast outputs, while the API reads these forecasts and writes the finalized ERP procurement decision records into the database.

3. Component References and Libraries Used

The system is built upon off-the-shelf components and standard libraries to ensure stability and maintainability:

- Mobile Interface (Java/Android SDK): MPAndroidChart
- Backend & API (Python): pyodbc, Flask / FastAPI, APIScheduler
- Machine Learning Engine (Python): pandas, scikit-learn, statsmodels, numpy, SimpleImputer, StandardScaler, TimeSeriesSplit, GridSearchCV, mean_absolute_error, mean_absolute_percentage_error, matplotlib

4. Data Structure Modeling (Entity Relationship Diagram)

Data architecture is a fundamental pillar that supports all system operations, from recording raw transaction logs to storing managerial decisions in the ERP module. At this design stage, all relational database schemas are modeled through an Entity Relationship Diagram (ERD) to define an organized storage structure in Microsoft SQL Server 2012. This ERD not only maps tables but also establishes data integrity rules through Primary Keys and Foreign Keys, ensuring that the data used by the Machine Learning prediction engine is consistent, valid, and traceable over time.

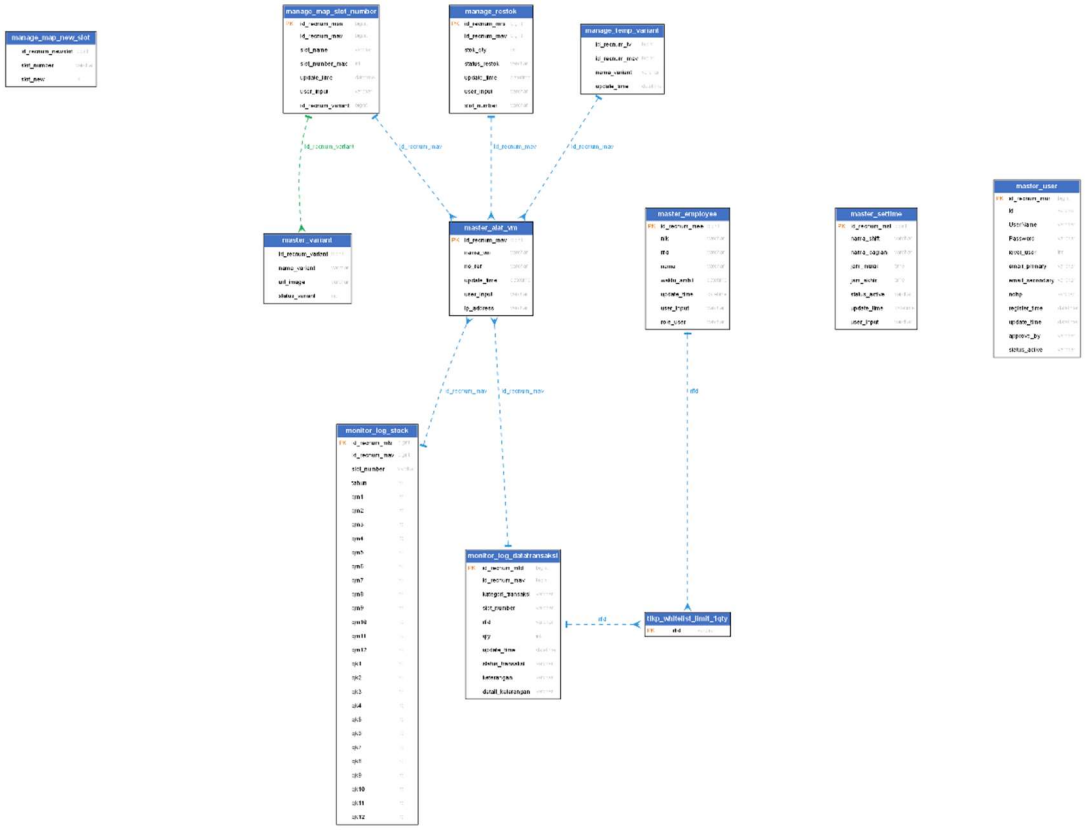

Figure 3.1, ERD diagram Database

C. STANDARDS USED

Category	Standard Name / Version	Application in the System
Architectural Style	REST (Representational State Transfer)	The standard architectural pattern used for building the Python Backend API to communicate with the Android application.
Data Exchange Format	JSON (JavaScript Object Notation)	The standard data format used for all HTTP request payloads and API responses between the Android client and the Python server.
System Modeling	UML 2.0 (Unified Modeling Language)	The standard used for designing system blueprints, including Use Case Diagrams, Activity Diagrams, and Sequence Diagrams.
Database Standard	ANSI SQL	The standard query language used to interact with the Microsoft SQL Server 2012 database for CRUD operations and data aggregation.
Time & Date Format	ISO 8601 (YYYY-MM-DD HH:MM:SS)	The standard datetime format used to record transaction logs from the dataset and to schedule the automated ML forecasting.
Code Formatting	PEP 8	The official style guide for Python code, applied to ensure readability and maintainability in the Backend API and Machine Learning scripts.
Network Protocol	HTTP / HTTPS	The standard application-layer protocol used for data transmission over the internal network.
ML Evaluation Standard	MAE & MAPE	The standard mathematical metrics (Mean Absolute Error & Mean Absolute Percentage Error) used to evaluate and validate the accuracy of the 3-month demand forecast model.

D. IMPLEMENTATION AND TESTING SCENARIO

The system is implemented as an ERP-based Decision Support System integrated with a Supervised Machine Learning module, using a client-server architecture consisting of a mobile application, backend API, machine learning engine, and centralized database.

1. End-to-End System Workflow & Implementation Flow

The system operates through an integrated pipeline, covering processes from data acquisition to decision-making, as follows:

a. Data Acquisition Layer (IoT Layer): Vending machines automatically generate transaction logs for each milk dispensing activity. The collected data includes a timestamp, machine ID, slot number, flavor, and quantity. These logs are periodically transmitted and stored in a SQL Server database, ensuring both real-time and historical data availability for analysis.

b. Data Prepocessing and Validation Layer (Backend Layer): The backend API performs data cleaning and validation to ensure data quality, including:
   - Removing duplicate and failed transactions
   - Handling missing values and inconsistencies
   - Validating slot-to-flavor mapping using master data

c. Data Aggregation and Feature Engineering Layer
   Transaction data is transformed into time-series datasets (daily or weekly), followed by feature engineering, such as: Lag consumption, Moving averages, Shift-based indicators, and Seasonality factors. These features enhance the model's ability to capture temporal patterns and consumption trends.

d. Machine Learning Forecasting Layer (Analytical Layer): Supervised machine learning models are used to generate demand forecasts, including: 3 month demand predictions, Flavor level demand breakdown, Model performance evaluation using metrics such as MAE and MAPE.

e. API Communication Layer: Communication between the mobile application and backend system is handled via REST APIs using JSON format. This ensures: Structured data exchange, Secure communication, and Real-time system responsiveness. The API can both retrieve processed data and trigger forecasting processes.

f. Presentation Layer (Mobile Application) : The Android application serves as the main user interface, providing:
   - Login (Authentication & RBAC)
   - Dashboard (consumption and stock trend monitoring)
   - Forecast Module (prediction visualization)
   - Procurement Planning (decision support form)

g. Decision Support Layer: The system automatically generates a draft procurement plan based on forecast results. However, final decision making remains with the user (GA Staff), who can: Review, Modify, Confirm the plan (RMC). This approach follows Decision Support System (DSS) principles, where technology supports rather than replaces human decisions. Final decisions are stored in the ERP system.

h. Security Implementation: System security is implemented through:
   - Role-Based Access Control (RBAC):
     1. GA Staff → limited access
     2. IT Admin → full system control
   - Anonymization of sensitive employee data in all visual outputs.

2. Testing Scenarios

A. Authentication and Access Control

No.	Feature	Scenario	Steps	Expected Result	Error Handling
1.	Login	Valid login	User inputs correct username & password → click login	User successfully enters dashboard	-
2.	Login	Wrong password	Input correct username + wrong password	Access denied	System displays: "Incorrect password"
3.	Login	Unregistered user	Input unknown username	Access denied	System displays: "User not found
4.	Login	Empty fields	User submits empty form	Login blocked	System displays: "Field cannot be empty"
5.	Access Control	Role restriction	GA accesses admin menu	Access denied	System displays: "Unauthorized access"

B. Dashboard Monitoring (Main UI)

No.	Feature	Scenario	Steps	Expected Result	Alternative / Error Handling
1.	Dashboard	Load dashboard	User opens dashboard	Charts displayed (daily, monthly, shift)	If no data → show "No data available"
2.	Dashboard	Filter data	Apply date/shift filter	Data updates dynamically	Invalid filter → reset automatically
3.	Dashboard	High data volume	Load large dataset	Data loads within acceptable time	Use aggregated/cached data
4.	Dashboard	Data inconsistency	Data anomaly exists	System still displays cleaned data	Outliers handled in backend

C. Data Processing & Validation (Backend)

No	Feature	Scenario	Steps	Expected Result	Alternative / Error Handling
1.	Data Input	Valid transaction	New vending data recorded	Data stored correctly	-
2.	Data Cleaning	Duplicate records	Duplicate detected	System removes duplicates	Log recorded for audit
3.	Data Cleaning	Missing values	Null data found	Data excluded or imputed	System flags invalid data
4.	Mapping	Slot mismatch	Slot not mapped to flavor	System checks master mapping	If not found → error log
5.	Data Integrity	Invalid format	Incorrect data format	Data rejected	Error message logged

D. Forecasting Module (ML UI)

No	Feature	Scenario	Steps	Expected Result	Alternative / Error Handling
1.	Forecast	Generate prediction	User opens forecast module	3-month forecast displayed	-
2.	Forecast	Flavor-level prediction	View detail forecast	Breakdown per flavor shown	-
3.	Forecast	Low data availability	Limited historical data	Forecast still generated	System shows warning: "Low data accuracy"

4.	Forecast	High anomaly data	Extreme spikes in data	Model handles preprocessing	Outliers removed
5.	Forecast	Accuracy metrics	View MAE/MAPE	Metrics displayed clearly	If high error → warning shown

E. Procurement Planning (Decision Support UI)

No.	Feature	Scenario	Steps	Expected Result	Alternative / Error Handling
1.	Planning	Auto-generate plan	System uses forecast	Draft procurement plan created	-
2.	Planning	Edit plan	GA modifies quantity	Changes saved successfully	-
3.	Planning	Save plan	User confirms plan	Plan stored in database	Confirmation popup shown
4.	Planning	No forecast data	Generate without forecast	Process blocked	System shows: "Forecast required"
5.	Planning	Overwrite plan	Existing plan updated	Data updated correctly	Require confirmation